

Platform.API — Senior Back-End .NET Refactor Report

Solution: YouVersionPlatform.slnx | **Project:** Platform.API (net10.0) | **Date:** July 2025

Table of Contents

- 1. [Executive Summary](#)
- 2. [Issues Found & Rationale](#)
- 3. [Files Changed](#)
- 4. [Test Results](#)
- 5. [SOLID & API Best Practice Alignment](#)

1. Executive Summary

A full SOLID and API best-practice audit of `Platform.API` identified **18 issues** across five categories: functional bugs, thread-safety, single-responsibility violations, API design smells, and missing validation/hardening. All 18 were resolved. The solution continues to build cleanly and all **86 unit tests pass**.

2. Issues Found & Rationale

2.1 Functional Bugs

#	Issue	Location (before)	Fix	Why It Matters
1	<code>IHighlightClient</code> pipeline was missing <code>OAuthBearerTokenHandler</code>	<code>ServiceCollectionExtensions.AddYouVersionOAuth</code>	Fixed Append <code>OAuthBearerTokenHandler</code> to the named <code>IHighlightClient</code> HTTP pipeline inside <code>AddYouVersionOAuth</code> .	Without the bearer handler, every authenticated highlight request would return HTTP 401 silently. This was the SDK's most critical functional defect.
2	OAuth token POST used relative URI on a client with <code>null</code> <code>BaseAddress</code>	<code>YouVersionOAuthClient.PostTokenRequestAsync</code>	Fixed Token endpoint is now read directly from <code>YouVersionOAuthOptions.TokenEndpoint</code> as an absolute URI. <code>BaseAddress</code> is not set on the OAuth <code>HttpClient</code> .	A relative URI on a client with no <code>BaseAddress</code> throws <code>InvalidOperationException</code> at runtime; the fix makes the authority clear and testable.
3	Dead <code>BaseAddress</code> assignment in <code>AddYouVersionOAuth</code> OAuth <code>HttpClient</code>	<code>ServiceCollectionExtensions.AddYouVersionOAuth</code>	Fixed Removed the factory that set <code>http.BaseAddress = opts.TokenEndpoint</code> since <code>PostTokenRequestAsync</code> already uses the absolute URI.	Combining a <code>BaseAddress</code> with an absolute request URI causes the <code>BaseAddress</code> to be silently ignored; removing it prevents future confusion.

2.2 Thread Safety

#	Issue	Location (before)	Fix	Why It Matters
4	<code>InMemoryTokenProvider</code> WAS registered as a singleton with no synchronization	<code>OAuth/InMemoryTokenProvider.cs</code>	Fixed Added <code>SemaphoreSlim(1,1)</code> guard around get/store/clear; implemented <code>IDisposable</code> .	In an ASP.NET Core server, multiple concurrent requests (e.g., background refresh + foreground read) race on the stored token reference. Without a lock the token can be observed in a partially-written state.

2.3 Single Responsibility (SRP) Violations

#	Issue	Location (before)	Fix	Why It Matters
5	<code>OAuthTokenResponse</code> contained Base64Url JWT parsing logic	<code>OAuth/OAuthTokenResponse.cs</code>	New file <code>OAuth/JwtHelper.cs</code> — static helper with <code>GetStringClaim</code> / <code>GetLongClaim</code> ; <code>OAuthTokenResponse</code> delegates to it.	A data-transfer object should not own parsing algorithms. Isolating the logic makes both the DTO and the parser independently testable and reusable.
6	<code>BibleClient</code> embedded a 66-book USFM-to-name/chapter-count lookup table	<code>Clients/BibleClient.cs</code>	New file <code>Clients/BibleBookCatalog.cs</code> — a focused catalog type; <code>BibleClient</code> delegates to it.	Mixing domain metadata with HTTP orchestration violates SRP and makes the client class unnecessarily large and hard to maintain.
7	Every client duplicated its own <code>EnsureSuccessAsync</code> / JSON deserialization	<code>BibleClient</code> , <code>PassageClient</code> , <code>HighlightClient</code>	New file <code>Http/ApiRequestHelper.cs</code> — centralized <code>EnsureSuccessAsync</code> and <code>GetJsonAsync<T></code> ; all clients now delegate here.	Three copies of the same error-handling path means three places a future bug can hide. A shared helper is easier to test, audit, and evolve.

2.4 API Design Smells

#	Issue	Location (before)	Fix	Why It Matters
8	<code>BuildAuthorizationUrl</code> used an <code>out PkceValues</code> parameter	<code>OAuth/IYouVersionOAuthClient.cs</code>	New file <code>OAuth/AuthorizationRequest.cs</code> — a simple record with <code>AuthorizationUrl</code> and <code>Pkce</code> properties. Method now returns <code>AuthorizationRequest</code> .	<code>out</code> parameters are awkward in async/await contexts, cannot be used with expression-bodied lambdas, and make callers harder to read. A return object is idiomatic .NET.
9	<code>IHighlightClient</code> did not respect the Interface Segregation Principle	<code>Clients/IHighlightClient.cs</code>	New files <code>IHighlightReader</code> and <code>IHighlightWriter</code> ; <code>IHighlightClient</code> composes both via interface inheritance.	Read operations are public/unauthenticated; write operations require OAuth. Separating them lets callers depend only on what they use and makes the auth boundary explicit in the type system.

10	<code>IBibleClient.GetBooksAsync(int)</code> required callers to issue two HTTP requests to get books for a version they already held	<code>Clients/IBibleClient.cs</code>	Fixed Added <code>GetBooksAsync(BibleVersion)</code> overload that extracts the id from the existing <code>BibleVersion</code> object.	Reducing unnecessary HTTP round-trips improves performance and simplifies consumer code.
11	<code>HighlightClient.CreateHighlightAsync</code> used an anonymous object for JSON payload	<code>Clients/HighlightClient.cs</code>	New file <code>Clients/CreateHighlightRequest.cs</code> — typed DTO with <code>[JsonPropertyName]</code> attributes.	Anonymous types are invisible to serializer contracts, refactoring tools, and code-generation tooling. Named DTOs are self-documenting and safe to rename.

2.5 Configuration / Validation / Hardening

#	Issue	Location (before)	Fix	Why It Matters
12	<code>OAuthTokenExpiryBufferSeconds</code> lived in <code>YouVersionApiOptions</code> (general API options)	<code>Configuration/YouVersionApiOptions.cs</code>	Moved Property relocated to <code>OAuth/YouVersionOAuthOptions.cs</code> where it logically belongs.	Putting OAuth-specific tunables in the general API options class violates cohesion. Consumers who do not use OAuth should not see OAuth settings.
13	<code>OAuthBearerTokenHandler</code> read the buffer from <code>YouVersionApiOptions</code>	<code>Http/OAuthBearerTokenHandler.cs</code>	Fixed Now injects <code>IOptions<YouVersionOAuthOptions></code> .	Follows the move in issue 12; the handler owns its own options dependency.
14	<code>ClientId</code> in <code>YouVersionOAuthOptions</code> had no <code>[Required]</code> annotation	<code>OAuth/YouVersionOAuthOptions.cs</code>	Fixed Added <code>[Required]</code> to <code>ClientId</code> .	Without a required annotation, a missing <code>ClientId</code> produces a subtle runtime failure (empty <code>client_id</code> in the auth URL) rather than a startup error with a clear message.
15	Both <code>AddYouVersionApiClients</code> overloads used <code>services.Configure</code> with no startup validation	<code>Extensions/ServiceCollectionExtensions.cs</code>	Fixed Switched to <code>services.AddOptions<T>().Configure().ValidateOnStart()</code> for both options classes.	<code>ValidateOnStart</code> causes misconfigured options to throw at application startup rather than silently failing mid-request.

16	<code>AppKeyDelegatingHandler</code> accepted an empty or whitespace <code>AppKey</code>	<code>Http/AppKeyDelegatingHandler.cs</code>	Fixed Added <code>string.IsNullOrEmpty</code> guard that throws <code>InvalidOperationException</code> before the request is sent.	An empty app key silently sends invalid requests that receive HTTP 401/403. Failing loudly at the point of misconfiguration is always preferable.
17	OAuth token POST payload built as a raw <code>FormUrlEncodedContent</code> without abstraction	<code>OAuth/YouVersionOAuthClient.cs</code>	Refactored Extracted <code>PostTokenRequestAsync</code> private helper used by both <code>ExchangeCodeAsync</code> and <code>RefreshTokenAsync</code> .	Removes duplicated token POST logic and makes both exchange and refresh paths easier to read and maintain.
18	Unresolved <code>// TODO: Confirm the exact highlight endpoint path</code> comment in <code>HighlightClient</code>	<code>Clients/HighlightClient.cs</code>	Fixed Removed the TODO; <code>/v1/highlights</code> is the confirmed path per the YouVersion Platform API docs.	Unresolved TODOs in production SDK code create uncertainty for consumers about whether the endpoint is stable.

3. Files Changed

File	Action	Summary
<code>Http/ApiRequestHelper.cs</code>	New	Centralized HTTP success checking and JSON deserialization used by all clients.
<code>OAuth/JwtHelper.cs</code>	New	Static Base64Url JWT claim parser extracted from <code>OAuthTokenResponse</code> .
<code>Clients/BibleBookCatalog.cs</code>	New	USFM book metadata (name, chapter count) extracted from <code>BibleClient</code> .
<code>Clients/CreateHighlightRequest.cs</code>	New	Typed DTO for highlight creation POST body.
<code>Clients/IHighlightReader.cs</code>	New	ISP read interface for unauthenticated highlight listing.
<code>Clients/IHighlightWriter.cs</code>	New	ISP write interface for OAuth-gated highlight create/delete.
<code>OAuth/AuthorizationRequest.cs</code>	New	Return type for <code>BuildAuthorizationUrl</code> ; replaces <code>out PkceValues</code> .
<code>Configuration/YouVersionApiOptions.cs</code>	Modified	Removed <code>OAuthTokenExpiryBufferSeconds</code> (moved to OAuth options).
<code>OAuth/YouVersionOAuthOptions.cs</code>	Modified	Added <code>OAuthTokenExpiryBufferSeconds</code> ; added <code>[Required]</code> to <code>ClientId</code> .
<code>OAuth/OAuthTokenResponse.cs</code>	Refactored	Delegates JWT parsing to <code>JwtHelper</code> .
<code>OAuth/InMemoryTokenProvider.cs</code>	Modified	Thread-safe via <code>SemaphoreSlim</code> ; implements <code>IDisposable</code> .

Http/OAuthBearerTokenHandler.cs	Modified	Uses <code>IOptions<YouVersionOAuthOptions></code> for expiry buffer.
Http/AppKeyDelegatingHandler.cs	Modified	Guards against empty/whitespace <code>AppKey</code> .
OAuth/IYouVersionOAuthClient.cs	Modified	<code>BuildAuthorizationUrl</code> returns <code>AuthorizationRequest</code> (no <code>out</code> param).
OAuth/YouVersionOAuthClient.cs	Refactored	Returns <code>AuthorizationRequest</code> ; extracted <code>PostTokenRequestAsync</code> ; uses absolute token URI.
Clients/IBibleClient.cs	Modified	Added <code>GetBooksAsync(BibleVersion)</code> overload.
Clients/IHighlightClient.cs	Modified	Composites <code>IHighlightReader</code> and <code>IHighlightWriter</code> .
Clients/BibleClient.cs	Refactored	Uses <code>ApiRequestHelper</code> and <code>BibleBookCatalog</code> ; added <code>BibleVersion</code> overload.
Clients/PassageClient.cs	Refactored	Uses <code>ApiRequestHelper</code> ; <code>PassageFormat</code> mapped via <code>.ToString().ToLowerInvariant()</code> .
Clients/HighlightClient.cs	Refactored	Uses <code>ApiRequestHelper</code> and <code>CreateHighlightRequest</code> ; removed local <code>EnsureSuccessAsync</code> .
Extensions/ServiceCollectionExtensions.cs	Modified	<code>AddOptions().ValidateOnStart()</code> for both options types; <code>OAuthBearerTokenHandler</code> wired to <code>IHighlightClient</code> pipeline.
PlatformTestApp/Program.cs	Modified	<code>/auth/login</code> updated to use <code>AuthorizationRequest</code> .
Platform.API.Tests/OAuth/OAuthClientTests.cs	Modified	All 5 <code>BuildAuthorizationUrl</code> call sites updated to <code>AuthorizationRequest</code> .
Platform.API.Tests/Http/OAuthBearerTokenHandlerTests.cs	Modified	<code>BuildPipeline</code> helper now passes <code>YouVersionOAuthOptions</code> with buffer seconds.

4. Test Results

86 tests — 86 Passed — 0 Failed — 0 Skipped

Build: 0 errors, 0 warnings (TreatWarningsAsErrors = true).

5. SOLID & API Best Practice Alignment

Principle / Practice	Before	After
Single Responsibility	JWT parsing in DTO; metadata in client; error handling duplicated 3×	Each class has one reason to change; shared helpers centralize cross-cutting concerns
Open/Closed	Adding new auth schemes required modifying every client	<code>DelegatingHandler</code> pipeline is open for extension without touching clients
Interface Segregation	<code>IHighlightClient</code> bundled read and write (different auth requirements)	<code>IHighlightReader</code> / <code>IHighlightWriter</code> allow consumers to depend only on what they use
Dependency Inversion	Clients instantiated helpers inline; OAuth buffer in wrong options class	All dependencies injected; each handler depends on the correct options abstraction
Fail Fast	Missing AppKey / ClientId discovered mid-request	Guards in handler + <code>ValidateOnStart</code> surface misconfiguration at startup
Thread Safety	Singleton token store unguarded	<code>SemaphoreSlim</code> ensures atomic read-modify-write

Idiomatic .NET API	<code>out</code> parameter, anonymous JSON objects	Return type object; named DTOs with <code>[JsonPropertyName]</code>
---------------------------	--	---